Overall Feedback

While each arrow has its corresponding relationship symbol, however adding the actual relationship helps to clear up any formatting differences. This can be applicable to arrows that do not use the default UML format and helps with the overall readability of the diagram, especially when multiple classes are interacting with one another.

Additionally, it is useful to both the reader and the programmer to see the input arguments and return types for each method/constructor. When multiple classes are interacting with one another, it should show how these classes relate. For example, if object A collects object B (aggregation) then it must show object B in the field. This improves readability as well as show the boundaries of the relationship.

Additional specific feedback is indicated in blue container below.

Suggestion: The game class uses pieces as objects, and will be destroyed when game object ends. To better capture this relationship a stronger from of association is better suited, possibly composition or aggregation.

Feedback: Since there are various different games that require the same features such as start game, take turn, and reset game, Game class should be abstract. By making the class abstract, other games can extend the class and tailor to their personal logic while keeping the functionality the same.

Feedback: Unclear how types of pieces are assigned, it would be ideal to use piece as an abstract class where there can be class that extend this class for each game. Additionally, enumeration can be used here where the piece constructor takes in an enum and sets its type.

Suggestion: checkWinCondition() and checkTieCondition() can be moved to GameLogic class where it is more appropriate. Rather than each game having different WinCondition(), it is better suit to have extensions of GameLogic since game for the most part are similar, however logic is different.

Feedback: Unclear how multiple players will interact with the board, the diagram only shows a single player interaction. May use queues along with sequence assigning to ensure each player gets their turn. Ideally this would have one to many relationship with the game since multiple players are active during game play.

Suggestion: The game class uses pieces as objects, and will be destroyed when game object ends. To better capture this relationship a stronger from of association is better suited, possibly composition or aggregation.

Feedback: Composition with multiplicity is not ideal here since user can have one curated profile. Having multiple profiles does not enhance the functionality of the application and can instead create new logins. In this case the multiplicity should be removed to have 1...1 relationship.

Suggestion: Display methods should be part of the GUI rather than the back end portion of the profile. However, methods such as displayStats() can return a string of data that can be passed to the GUI to be printed, which would require the method to have a return type of string.

Feedback: There is conflicting information regarding the rank, player class on the right it shows that rank is a enum while here it shows as a string. Having consistency across the diagram is key to ensure all classes interact with other classes in the same manner.

Suggestion: EmailService should be connected to the networking class to ensure that the class is utilized appropriately. Without having any network communication it is difficult to transmit data for confirmation and resetting password.

Suggestion: EmailService will require authentication to prove the email exists, and/or any other proof of identity to ensure the user is authorized to change credential. This can either be integrated within the class using helper methods, or can use AuthenticationService in which case relationship be reversed.

Suggestion: viewCurrentProfile() can be connected to user stats that shows all the details. Rather than having a method that performs similar functions as a class it is beneficial to use the UserStats with an association.

Suggestion: CurrentStatus can be changed into a enum which will be set when creating the profile or toggled by the user. Having a string type is not ideal when setting the status for accessiblity and may require additional error checking.

Suggestion: Rank is set in the profile, rather than having a field with the same attribute, it is best to reduce redundancy. Can also implement a method in user that can get all field attributes to set userstats.

Suggestion: GameHistory is stored into an arraylist, which can be better captured using association with multiplicity. This shows that these objects will tracked simultaneously.

Feedback: AuthenticationHandler should have an association with User to ensure that when user creates profile, the login data is prepared and authenticated. Ideally, service handles this property which is counter intuitive since Handler prepares login data.

Feedback: ClientConnection class does not appear on the class structure diagram including the documentation for java standard libraries. To ensure readability and clarity, ensure all classes are documented on the diagram.

Comment: Java.net, java.NIO, and java.lang.Thread are java standard libraries that used in the project. The representation of these libraries is shown appropriately and clearly indicates that these libraries will be associated with the Networking class.

Feedback: LeaderBoardEntry is not a class represented on this class structure diagram. It is unclear how this class interacts and what corresponding details this class contains. Ensure all classes that are used are shown on the diagram.

Suggestion: Leaderboard can benefit from an rank comparator class that will decide the sequence that the players are ordered. The comparator method is useful to add complex behaviour that may apply to the rank, such as skill based match making rank or game pieces moved to win. These attributes can determine the leaderboard in greater detail to show how one player stacks up to another. Furthermore, it can be coupled with the "equal" method under the player class to ensure consistency.

Feedback: Remove association arrow. Association here is unnecessary since composition is a stronger association relationship therefore is implicitly denoted.

Feedback: Networking class acts as the hub to connect and control all of the components within the project. This way the networking class owns ChatManager, GameSyncManager, and AuthenticationHandler using an composition relationship. In this case all of the composition arrows must be reversed to ensure the correct relationship is communicated.

Feedback: BoardState is missing from the class structure diagram. To ensure that relationships and interactions are clear, documentation for these classes is necessary.

Suggestion: Matchmake should have many to one relationship with player to show that multiple player objects are being collected when interacting with the queue. This relationship aligns with the UML format to ensure readability and show the boundaries of the relationship.

Suggestion: Unless the placement object is recreated for each instance the player is in matchmaking it would be ideal to add reset placement test to remove previous results. This way the object can be reused and be optimized better for the application.

Suggestion: Player should have an association relationship with multiplicity since leader board tracks multiple player objects and are stored in a custom list. This relationship makes it to the programmer and reader that all methods of the player class are accessible when updating the leaderboard in their respective methods.

Suggestion: Composition arrow should be reversed, player class owns rank objects. This way when player object is destroyed the rank (enumeration) object is destroyed alongside.

Suggestion: Unless this instance of the player class is under its own package, there will be a naming conflict with a player class that interacts with the game board. Ensure class naming is handled appropriately to eliminate any conflicts.

Suggestion: UpdatePlayerTTT, updatePlayerCheckers, updatePlayerC4 should be moved in to their respective classes rather than in the super class. This ensures that each instance class does not have to implement methods that are irrelevant to those classes.

Suggestion: LeaderBoard should have an abstract label that displays that the class cannot be instantiated and needs to be extended through personalized classes (as shown below).

Suggestion: Ideally enumeration classes are used to set status by a predefined value. These ranks should be upper caps to align with format of constants. Additionally, these ranks can have predefined value assignment for easier tracking and comparison.

Suggestion: As discussed in the game logic section, ideally these games can extend the game class, given game class is an abstract class. This would allow for similar functionality while modifying the game logic individually.

Feedback: ChatManager is a superclass that tracks and manages the Chatbox GUI, therefore the association arrow should be reversed. This ensures that ChatManager handles the requests sent by the Chatbox such as displayMessages.

Suggestion: AnimatedLoginScreen should be associated with the networking class since the data from the text fields is sent to be authenticated. Without having this relationship, the AuthenticationHandler and Service cannot interact, therefore should be appropriately handled to ensure a seamless transfer of data.

Suggestion: AnimatedLoginScreen should account for other components that are used such as alerts that can be displayed for error handling and listener objects that that track user inputs.

Suggestion: All of the components that are dependencies of MainGamePlatform require the JavaFX library import. This diagram will benefit from showing this library, since most GUI is based around these library components.

Feedback: Arrows should be moved towards the centre of the class container. With this current layout, it is unclear where each of these arrows are leading to and from. This reduces readability and must make assumptions about the diagram.

Feedback: Association arrow should be reversed, LobbyPage is the superclass and ChatBoxGUI will be apart of the lobby as an additional function.

Feedback: Unclear how different variations of the game will be displayed. PlayGame can be an abstract class which the individual games can have classes and override the methods defined in PlayGame.

Suggestion: The JavaFX program can benefit from a debugging/console log output that records any errors or success messages. This ensures that user and programmer can identify the issue that may be caused and not caught by other methods. It can work well with the MainGamePlatform which is the main JavaFX executable code, since launching may cause crashes that are difficult to debug. This output file can be created everytime the application is launched and/or when the user executes a specific step, and previous file can be overwritten.

Piece
- type
- color
+ getType()
+ getColor()
+ promote()

Game
- currentPlayer
- gameState
+ startGame()
+ takeTurn()
+ checkWinCondition()
+ checkTieCondition()
+ resetGame()

0..*

1

Player
- symbol
+ makeMove(position)

Board
- grid
+ initialBoard()
+ getPieceAt (position)
+ setPieceAt(position)
+ setPieceAt(position, piece)
+ isFull()

GameLogic
+ checkVictory()
+ checkDraw()
+ switchPlayer()

EmailService
+ sendConfirmationEmail(email: String): boolean
+ sendPasswordResetEmail(email: String): boolean
+ sendDeletedProfileEmail(email: String): boolean

AuthenticationService
+ validateCredentials(username: String, password: String): boolean
+ resetPassword(email: String): boolean

User
- userID: int
- username: String
- password: String
- email: String
- profile: Profile
- rank: String
+ login(username: String, password String): boolean
+ logout(): void
+ registerNewUser(userName: String, password: String, email: String): void
+ updateUsername(newUsername: String): void
+ updateEmail(newEmail: String): void
+ viewCurrentProfile(): Profile

1..*

Profile
- userID: int
- username: String
- gamesPlayed: ArrayList<GameHistory>
- userStats: UserStats
- currentStatus: String
- record: Record
+ displayStats(): void
+ viewGameHistory(): void
+ updateStats(newStats: UserStats, newGames: GameHistory: newRecord: Record): void
+ searchProfile(username: String): Profile

Record
- totalWins: int
- totalLosses: int
- totalDraws: int
+ getRecordDetails(): void
+ updateGameRecord(): void

UserStats
- totalGamesPlayed: int
- wins: int
- losses: int
- draws: int
- rank: String
+ getRankDetails(): void
+ updateRank(newRank: String): void
+ calculateWinRatio(): double

GameHistory
- gameID: int
- gameName: String
- datePlayed: int
- opponent: String
- score: int
- outcome: String
+ displayGameHistory(): void

Matchmake
- playerQueue: Queue<Player>
+ addPlayerToQueue(player: Player): void
+ removePlayerFromQueue(player: Player): void
+ returnBestMatch(playerQueue: Queue<Player>): ArrayList<Player>
+ canMatchMake(playerQueue: Queue<Player>): boolean
+ clearQueue(): void

PlacementTest
- round1: int
- round2: int
- round3: int
- round4: int
- round5: int
+ PlacementTest (player: Player)
+ setScore (round: int, score: int) :void
+ setRank (gameplayed: int): void

Player
- username: String
- id: int
+ placementGamesLeft: int
- rank: enum Rank
+ equals(obj: Object): boolean

0..*

1

<<enumeration>> Rank
Iron
Bronze
Silver
Gold
Platinum
Diamond
Immortal
Radiant

LeaderBoard
players: List <Player>
+ addPlayer(player: Player): void
+ updatePlayerC4(player: Player, won: boolean): void
+ updatePlayerTTT(player: Player, won: boolean): void
+ updatePlayerCheckers(player: Player, won: boolean): void
+ recalculateMMR (player: Player, game: int): void
+ displayLeaderBoard(): void

AdjustRating
- minimum_mmr_gain: int
- minimum_mmr_loss: int
- mmrChange: int
+ updateRating(): void
+ updateRank(): void

CheckersLB
+ displayLeaderBoard(): void

TicTacToeLB
+ displayLeaderBoard(): void

ConnectFourLB
+ displayLeaderBoard(): void

+ AuthenticationHandler
- username: String
- password: String
+ prepareLoginData(username: String, password: String)
+ prepareRegData(username: String, password: String)
+ receiveAuthResponse(response: String)
+ checkAuthentication(): boolean

+ ServerRelay
- connectedClients: List<ClientConnection>
- matchQueue: Queue<ClientConnection>
+ authenticateClient(client: ClientConnection): boolean
+ matchClients()
+ relayData(sender: ClientConnection, receiver: ClientConnection, data: String)

+ LeaderboardHandler
- leaderboardData: List<LeaderboardEntry>
+ requestLeaderboard()
+ updateLeaderboard(data: List<LeaderboardEntry>)
+ saveLeaderboardData()

+ Networking
- msgToBeSent: String
- connectionStatus: Boolean
+ startComponents()
+ runLoop()
+ requestGame(gameType: String)
+ sendData(data: String)
+ receiveData(): String

Java.net

java.NIO

java.lang.Thread

+ ChatManager
- messageQueue(): Queue<Message>
- msgToBeSent(): Message
+ sendMessage(msg: Message)
+ receiveMessage(): Message
- decodeMessage(msg: Message)

+ GameSyncManager
- boardState: BoardState
+ sendBoardState(boardstate: BoardState)
+ receiveBoardState(): BoardState
+ endUserTurn()
+ setUserTurn(playerID: int)

+ sendBoardState(boardstate: BoardState)
+ receiveBoardState(): BoardState
+ endUserTurn()
+ setUserTurn(playerID: int)

For sound effect

+ Media
- audioFiles: List<String>
+ playsound(): String
+ soundTransition()
+ stopSound(): Boolean
+ soundSpeed() : int

+ AnimatedRegistrationScreen
- registration: Form
- storage: Query
+ displayRegistration()
+ displayRegistrationMessage()
+ lobbyTransition()

+ AnimatedLoginScreen
- usernameTextbox: TextField
- password Textbox: TextField
- backgroundPicture : String
+ displayLoginScreen()
+ authentication(String, String)
+ displayLoginMessage()

+ MainGamePlatform
- availableGames: List<String>
+ displayGames()
+ chooseGame()
+ choosePlayers()
+ notifications()

+ Chatbox GUI
- currentPlayers() : Player
+ displayMessages()
+ displayChatbox()

+ PlayGame
- board: GridPane
+ displayInstructions()
+ displayGameBoard()
+ displayLeaderBoard()
+ displayWinner()

+ LobbyPage
- currentPlayers() : Player
+ displayStatusMessage(String): String